Project Interim Report

On

AYURVEDIC HERB IDENTIFICATION

(Using Python & ML)

RU-500-02-00002

Foundations of Machine Learning

SESSION: 2025-26

Submitted By

KUMARI AASHI

RU-25-10717

Master of Computer Application in AI and ML in Association with Google

Under the Guidance of

Mr. Zamin Hamid

SCHOOL OF COMPUTER INFORMATION TECHNOLOGY

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

PlantLens Web is a Flask-based plant and Ayurvedic herb identification web application. The system enables users to upload a plant photograph or capture one directly using the browser camera, then sends the image to the PlantNet API for species identification. The result is enriched using multiple data sources including Wikipedia, Wikidata, Perenual, and a local Ayurveda dataset, and presented in a clean, informative result page.

Unlike standard plant identification tools, PlantLens Web integrates traditional Ayurvedic knowledge, providing Sanskrit names, dosha effects, medicinal benefits, and traditional use for identified plants. The application helps users understand not just what a plant is, but also its regional names, safety profile, and medicinal value — making it a unique educational tool.

Introduction

Agriculture, botany, and traditional medicine are deeply connected in Indian culture. Yet many people encounter plants daily without knowing their name, scientific classification, safety concerns, or medicinal significance. The gap between visual observation and botanical knowledge is the core problem this project addresses.

PlantLens Web is designed to bridge this gap by combining modern AI-based image recognition with curated educational data. The application uses the PlantNet API — a large-scale plant identification engine — to recognize the plant from an image. Once identified, it gathers complementary data from Wikipedia for descriptions, Wikidata for local Indian language names, Perenual for toxicity data, and a locally maintained Ayurveda dataset for traditional medicinal context.

Existing plant identification applications typically display only the plant's scientific name and a basic description. PlantLens Web extends this significantly by adding regional names in Hindi, Tamil, Telugu, Marathi, and Sanskrit, along with Ayurvedic properties such as dosha balance, benefits, and traditional usage. This makes the application particularly relevant for students, educators, herbalists, and curious general users in India.

Key Features

- Upload a plant image from the device
- Drag and drop an image onto the upload area
- Capture a photo directly using the browser camera
- Compress uploaded images before sending to the identification API
- Identify the plant using PlantNet API
- Show top 3 possible plant matches with confidence scores
- Display confidence score as a percentage and visual progress bar
- Fetch plant summary and Wikipedia link
- Fetch local/regional names from Wikidata (Hindi, Tamil, Telugu, Marathi, Sanskrit)
- Fetch toxicity information for humans and pets from Perenual
- Show Ayurvedic properties from a local JSON dataset (44 plants)
- Maintain last 20 scans in browser session history
- Allow clearing scan history
- Render deployment configuration included

Scope

PlantLens Web is suitable for botany students, Ayurveda practitioners, agriculture advisors, and general users curious about plants and their medicinal properties. The application is built as an educational and demonstration tool and is not intended for professional medical or botanical diagnosis.

The system can be extended in future by integrating real-time plant databases, mobile applications, user login systems, and multilingual UI support. The Ayurveda dataset can be expanded beyond its current 44 entries, and cached API responses could significantly improve performance at scale.

Project Summary

Item	Details
Project Name	PlantLens Web
Main Goal	Identify plants from images and show botanical plus Ayurvedic information
Backend	Python Flask
Frontend	Jinja2 HTML templates, TailwindCSS CDN, custom CSS, vanilla JavaScript
Main AI/API Feature	Plant recognition through PlantNet API
Extra Data Sources	Wikipedia REST API, Wikidata SPARQL, Perenual API, local Ayurveda JSON
Storage	Flask signed session for recent scan history, uploaded images on local disk
Deployment Target	Render using Gunicorn

System Requirements

Hardware

- Computer with minimum 4 GB RAM
- Internet connection for API access
- Camera access for browser-based photo capture (optional)

Software

- Python 3.8 or higher
- Visual Studio Code or any code editor
- Libraries: Flask, python-dotenv, requests, Pillow, Gunicorn
- Web browser with HTTPS support for camera capture

Methodology

The system operates through a well-defined request-response pipeline. The following steps describe the full data flow from user action to displayed result:

1. User opens the home page and sees the upload form, drag-and-drop area, and camera button.
2. User selects an image file, drags and drops one, or captures a photo using the browser camera via navigator.mediaDevices.getUserMedia().
3. The form submits a POST request to the Flask route /identify with the image file.
4. Flask saves the image to static/uploads/ using a UUID-based filename to prevent overwriting.
5. The image is compressed using Pillow: maximum dimension 800 pixels, RGB conversion if needed, JPEG quality 85.
6. The compressed image is sent to the PlantNet API which returns up to 3 species matches with confidence scores.
7. The top match is selected and enriched: Wikipedia provides description and URL, Wikidata provides regional names, Perenual provides toxicity flags, and the local Ayurveda JSON provides medicinal details.
8. A lightweight scan summary is saved in Flask session (maximum 20 scans stored).
9. The result.html template renders all collected data in a structured, readable page.

High-Level Architecture

The application follows a Flask MVC-style structure with clear separation of concerns across four layers:

- Routes layer (routes/): URL endpoints and request handling logic
- Services layer (services/): External API and data lookup logic
- Utilities layer (utils/): Reusable helper functions such as image compression
- Templates/Static layer (templates/, static/): Jinja2 HTML pages, CSS, and JavaScript

Architecture Flow:

User Browser → POST /identify → Image Save & Compress → PlantNet API → Enrichment Services (Wikipedia, Wikidata, Perenual, Local JSON) → result.html → Session History

Folder Structure

```
web/
  app.py
  requirements.txt
  render.yaml
  .env
  data/
    ayurveda_plants.json
  routes/
    identify.py
    history.py
  services/
    plantnet.py
```

```
    wikipedia.py
    perenual.py
    ayurveda.py
  static/
    css/style.css
    js/upload.js
    js/camera.js
    uploads/
  templates/
    base.html
    index.html
    result.html
    history.html
  utils/
    image_utils.py
```

External APIs Used

API / Service	Purpose	Key Required?
PlantNet API	Identify plant species from an uploaded image	Yes (PLANTNET_API_KEY)
Wikipedia REST API	Get readable plant description and page URL	No
Wikidata SPARQL	Search for local and regional plant names	No
Perenual API	Check if plant is poisonous to humans or pets	Yes (PERENUAL_API_KEY)
Local Ayurveda JSON	Ayurvedic properties (44 plant entries)	No (local file)

Local Ayurveda Dataset

File: data/ayurveda_plants.json

The dataset currently contains 44 plant entries, each keyed by scientific name. Each entry may contain the following fields:

- sanskrit_name: Traditional Sanskrit name of the plant
- local_name_hi: Common Hindi name
- dosha: Ayurvedic dosha balance effect (Vata, Pitta, Kapha)
- benefits: List of health and medicinal benefits
- properties: Taste, potency, and other classical properties
- traditional_use: Cultural and historical use in Ayurvedic practice

Example entry for Tulsi (Ocimum tenuiflorum):

Sanskrit Name: Tulasi | Dosha: Balances Vata and Kapha | Benefits: Boosts immunity, Relieves respiratory issues | Properties: Bitter, pungent taste; heating potency | Traditional Use: Sacred plant in Hindu tradition

This local dataset is a core differentiator of the project because no reliable public API provides structured Ayurvedic data for all plants. Keeping it local ensures speed, control, and academic suitability.

Implementation

Backend Classes and Files

- app.py: Main Flask entry point. Loads environment variables, registers blueprints, sets upload limit to 10 MB.
- routes/identify.py: Main scan route (POST /identify). Connects upload, compression, PlantNet, enrichment, session history, and rendering.
- routes/history.py: Handles GET /history and POST /history/clear for session-based scan history.
- services/plantnet.py: PlantNet API wrapper. Sends image, parses results, returns list of plant match dictionaries.
- services/wikipedia.py: Fetches plant description from Wikipedia summary API and regional names from Wikidata SPARQL.
- services/perenual.py: Fetches toxicity flags (poisonous_to_humans, poisonous_to_pets) from Perenual API.
- services/ayurveda.py: Loads Ayurveda JSON and performs exact or genus-level scientific name lookup.
- utils/image_utils.py: Compresses images using Pillow — max 800px, RGB conversion, JPEG quality 85.

Frontend Files

- templates/base.html: Shared layout with TailwindCSS CDN, Google Fonts, navbar, footer, script block.
- templates/index.html: Home page with upload form, drag-and-drop, image preview, camera modal, identify button.
- templates/result.html: Result page showing top match, confidence, local names, description, safety, Ayurvedic info, and other matches.
- templates/history.html: History page showing scan cards with thumbnails, names, and confidence badges.
- static/js/upload.js: Handles file selection, drag-and-drop, preview, validation, and submit loading state.
- static/js/camera.js: Opens camera modal, captures video frame to canvas, converts to JPEG, assigns to form input.
- static/css/style.css: Custom styles including drag-over highlight, spinner animation, scrollbar, and image preview transitions.

Sample Input and Output

Sample Input

- Image: Photograph of a Tulsi plant (uploaded or captured)
- Method: Browser camera or file upload

Sample Output

- Common Name: Tulsi
- Scientific Name: Ocimum tenuiflorum
- Family: Lamiaceae
- Confidence Score: 93.2%
- Wikipedia Description: Tulsi is a sacred aromatic plant in Hindu culture...
- Local Names: Tulasi (Hindi), Thulasi (Tamil), Tulasi (Sanskrit)
- Toxicity: Not poisonous to humans. Not poisonous to pets.
- Ayurvedic Benefits: Boosts immunity, relieves respiratory issues
- Dosha: Balances Vata and Kapha

Environment Variables

The application expects a .env file inside the web/ directory with the following keys:

- PLANTNET_API_KEY: Allows plant image identification via PlantNet
- PERENUAL_API_KEY: Allows toxicity lookup via Perenual
- SECRET_KEY: Signs Flask session cookies for security

The .env file must not be committed to Git as it contains secret credentials.

Dependencies

- flask==3.0.3: Web framework
- python-dotenv==1.0.1: Loads environment variables from .env
- requests==2.31.0: Makes HTTP calls to external APIs
- pillow>=11.0.0: Image compression and format conversion
- gunicorn==22.0.0: Production WSGI server for Render deployment

Project Timeline

Week 1: Planning and Setup

- Requirement analysis and feature scoping
- API key registration (PlantNet, Perenual)
- Flask project structure setup
- Environment configuration and .env setup

Week 2: Core Backend Development

- Implement PlantNet API integration (services/plantnet.py)
- Implement Wikipedia and Wikidata services
- Implement Perenual toxicity service
- Build image upload and compression pipeline

Week 3: Dataset and Enrichment

- Build and populate Ayurveda JSON dataset (44 entries)
- Implement Ayurveda service with exact and genus-level lookup
- Implement session-based scan history
- Connect all services in routes/identify.py

Week 4: Frontend, Testing, and Deployment

- Build Jinja2 templates (index, result, history pages)
- Implement drag-and-drop and camera capture with JavaScript
- Style with TailwindCSS and custom CSS
- Test with real plant images and fix edge cases
- Configure Render deployment with render.yaml
- Prepare documentation and project report

Current Limitations

10. Environment variable loading order issue: route modules are imported before load_dotenv() in app.py, which may cause API key values to be empty unless set in shell.
11. API errors are caught silently — users may see 'Unknown' instead of helpful error messages.
12. History is session-based only; it is lost when the session changes or deployment storage resets.
13. Uploaded image files are not automatically cleaned up from static/uploads/.
14. The 'Use this' button for alternate plant matches displays an alert instead of reloading enriched data for that match.
15. The 'View Details' button on history cards shows an alert and does not reopen the full saved result.
16. The Origin & Habitat card on the result page shows fallback text because get_plant_details() does not return an origin field.

Strengths of the Project

- Clear MVC-style separation: routes, services, templates, static files, and utilities are all independent.
- Multiple data sources are combined into one comprehensive educational result page.
- Server-side API calls protect all secret keys from being exposed to the frontend.
- Image compression improves upload speed and reduces external API processing time.
- User interface supports both file upload and live camera capture workflows.
- Local Ayurveda dataset makes the project unique compared to standard plant identification apps.
- Render deployment configuration is already included in the codebase.
- Codebase is small and well-structured, suitable for clear college presentation.

Future Improvements

- Fix environment variable loading order by moving load_dotenv() before all imports in app.py.
- Add user-facing error messages when external APIs fail.
- Store full scan results in a database for permanent history.
- Add user login so scan history persists across devices and sessions.
- Implement automatic cleanup of old uploaded image files.
- Make the 'Use this' alternate match button fully functional.
- Make the 'View Details' button in history open a complete saved result page.
- Cache Wikipedia, Wikidata, and Perenual responses to reduce redundant API calls.
- Expand the Ayurveda dataset from 44 to 200+ plant entries.
- Add Hindi and other regional language UI support.
- Add a confidence threshold warning for low-confidence scan results.
- Add a disclaimer before displaying medicinal benefits.

Conclusion

PlantLens Web successfully demonstrates the practical application of Flask web development, API integration, and data enrichment techniques in a real-world context. The project addresses a genuine knowledge gap by combining modern plant recognition technology with traditional Ayurvedic wisdom.

The system identifies plants from photographs, enriches the result with multiple reliable data sources, and presents the information in a structured, educational format accessible to any user through a browser. The use of a local Ayurveda dataset, regional Indian language names, and toxicity information distinguishes the project from standard plant identification tools.

The main technical contribution is the integration pipeline: a single image input triggers a coordinated sequence of API calls and data lookups that together produce a comprehensive, multi-dimensional result. This architecture demonstrates clean separation of concerns, secure API key management, and responsive frontend design.

References

17. PlantNet API Documentation. Available: https://my-api.plantnet.org
18. Wikipedia REST API. Available: https://en.wikipedia.org/api/rest_v1/
19. Wikidata SPARQL Endpoint. Available: https://query.wikidata.org/
20. Perenual API Documentation. Available: https://perenual.com/docs/api
21. Flask Documentation. Available: https://flask.palletsprojects.com/
22. Pillow (PIL Fork) Documentation. Available: https://pillow.readthedocs.io/
23. TailwindCSS Documentation. Available: https://tailwindcss.com/docs